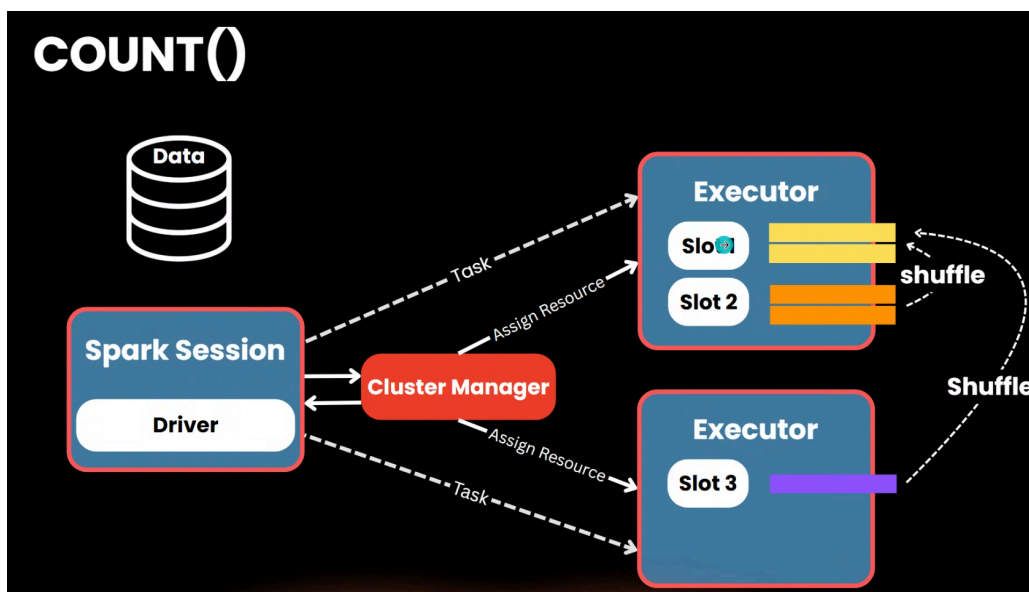


Question & Highlight Module 6

- ทำไมยังมีคนใช้ pandas แทนที่จะใช้ PySpark ทั้งหมด
คำตอบอยู่ที่ library ที่ support ใน pandas เช่น กราฟ, numpy
และบางบริษัทไม่ได้มีจำนวน data เยอะ ขนาดที่ต้อง setup spark
- Type Safety
 - คือ คุณสมบัติของภาษาโปรแกรมหรือระบบที่ช่วยป้องกันไม่ให้เกิดความผิดพลาดจากการใช้ "ประเภทข้อมูล" (Data Type) ผิดประเภทครับ
 - **Compile-time (ก่อนรันโปรแกรม):** เป็นระดับที่ปลอดภัยที่สุด ตัวแปลภาษา (Compiler) จะตรวจเช็คให้เลยว่าคุณส่งค่า String ไปใส่ในตัวแปรที่เป็น Integer หรือเปล่า ถ้าผิดมันจะไม่ยอมให้คุณรันโปรแกรม
 - **Runtime (ขณะรันโปรแกรม):** ถ้าภาษาไม่มี Type Safety ที่ดีพอ มันจะปล่อยให้ผ่านไปได้ แต่จะไป "ระเบิด" หรือ Error กลางคันขณะที่ผู้ใช้งานกำลังใช้งานอยู่



คำศัพท์เพิ่มเติม

ในกระบวนการทำงานจริง ข้อมูลจะไหลจาก **Source** ผ่าน **Slots** ใน **Nodes** ต่างๆ และอาจเกิด **Shuffle** ก่อนจะไปสิ้นสุดที่ **Sink** ครับ

1. Read (Source → Slot):

- Executor ในแต่ละ **Node** จะสั่งให้ **Slot** (หน่วยประมวลผลระดับ CPU Core) รุ่งไปอ่านข้อมูลจาก **Source**
- ข้อมูลจะถูกแบ่งเป็น **Partition** โดย 1 Slot จะรับผิดชอบประมวลผล 1 Partition เสมอ

2. Shuffle (Slot ↔ Slot):

- ถ้าเป็นการประมวลผลแบบ **Narrow** ข้อมูลจะจบใน Slot นั้นๆ เลย
- แต่ถ้าต้องทำ **Shuffle** (เช่น `join` หรือ `groupBy` ตามที่คุณสรุป) ข้อมูลจะถูกเขียนลง Disk ชั่วคราว (Shuffle Write) แล้วส่งผ่าน Network ไปยัง **Slot** ในเครื่องอื่น (Shuffle Read) เพื่อรวมกลุ่มตาม Key เดียวกัน

3. Write (Slot → Sink):

- เมื่อคำนวณเสร็จสิ้น ผลลัพธ์สุดท้ายจากแต่ละ Slot จะถูกเขียนออกไปที่ **Sink** พร้อมกันแบบขนาน (Parallel Write)

1. Narrow Transformation (การแปลงแบบ "แคบ")

คำว่า "**Narrow**" (แคบ) หมายถึง ข้อมูลจาก 1 Partition ฝั่ง Input จะไหลไปสร้าง 1 Partition ฝั่ง Output (หรือน้อยกว่า) โดย **ไม่ต้องพึ่งพาข้อมูลจาก Partition อื่นเลย**

- **ลักษณะ:** เหมือนวิ่งในเลนตัวเอง (One-to-One)
- **ทำไมถึงเรียกแบบนี้:** เพราะขอบเขตการทำงานมัน "แคบ" อยู่แค่ภายในเครื่องเดียว หรือ Partition เดียว ไม่ต้องไปยุ่งกับเครื่องอื่น
- **ตัวอย่าง:** `map()`, `filter()`, `flatMap()`
- **ข้อดี:** เร็วมาก เพราะไม่ต้องเกิด **Shuffle** (ไม่ต้องส่งข้อมูลข้าม Network)

2. Wide Transformation (การแปลงแบบ "กว้าง")

คำว่า "**Wide**" (กว้าง) หมายถึง ข้อมูลจาก 1 Partition ฝั่ง Input อาจจะต้องถูกกระจายไปอยู่ใน **หลายๆ Partition** ของฝั่ง Output เพื่อให้ได้ผลลัพธ์ที่ต้องการ

- **ลักษณะ:** มีการตัดสลับเลนไปมา (Many-to-Many)
- **ทำไมถึงเรียกแบบนี้:** เพราะขอบเขตการทำงานมัน "กว้าง" ข้ามไปทั้ง Cluster ข้อมูลจาก Partition A อาจจะต้องไปรวมกับข้อมูลจาก Partition B, C, D ที่อยู่คนละเครื่องกัน
- **ตัวอย่าง:** `groupByKey()`, `reduceByKey()`, `join()`, `repartition()`

- **ผลกระทบ:** ทำให้เกิด **Shuffle** ซึ่งเป็นตัวการที่ทำให้ Job รันช้าลงและกินทรัพยากรเครื่องสูง

```

1  (
2    read_csv_df
3    .withColumn("shop_id"
4      ,when(col("shop_id") == 1,lit("20"))
5      .when(col("shop_id") == 2,lit("30"))
6      .otherwise(col("shop_id")))
7    .filter(
8      ~((col("shop_id") == "30") | (col("shop_name") == "valen")))
9    )
10   ).display()
> See performance \(1\)

```

- filter "&", "|" ต้องระวังเรื่อง การใส่ () ครอบด้วย ไม่งั้นจะ error

```
~((col("shop_id") == "30") | (col("shop_name") == "valen"))
```

- .explain()
 - explain(mode= "formatted")
 - เป็นโหมดที่ถูกเพิ่มเข้ามาเพื่อให้ **Data Engineer** อ่านง่ายขึ้นมาก โดยจะจัดระเบียบข้อมูลใหม่ให้เป็นสัดส่วน
 - **สิ่งที่แสดง:**
 - **แบ่งเป็น Node:** แสดงลำดับการทำงานเป็นข้อๆ (Node 1, Node 2, Node 3...)
 - **สรุป Operator:** บอกชัดเจนว่าจุดไหนคือการ **Scan** (อ่านไฟล์), จุดไหนคือ **Filter** (กรองข้อมูล), และจุดไหนคือ **Project** (เลือกคอลัมน์)
 - **แสดง Schema:** บอกประเภทข้อมูล (Type) ที่ไหลผ่านในแต่ละขั้นตอนอย่างชัดเจน
 - **ข้อดี:** ช่วยให้เราเห็นภาพรวมของ Query ได้รวดเร็ว โดยไม่ต้องเพ่งตัวอักษรที่ติดกันเป็นพืด